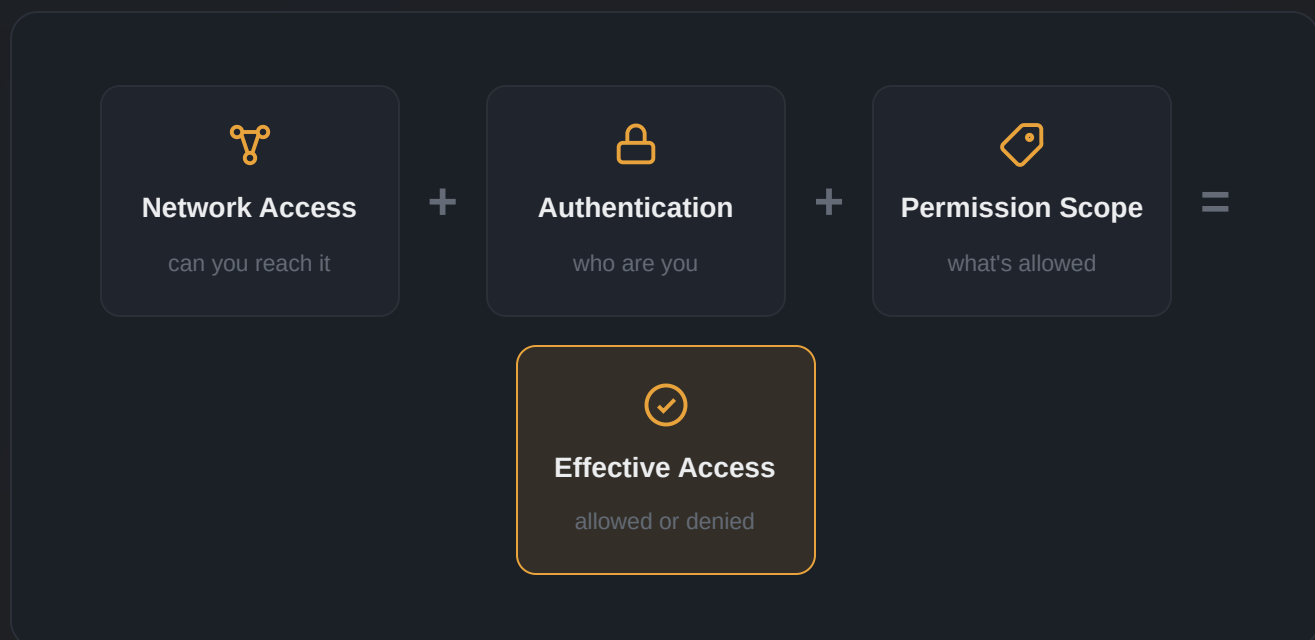


Configure Access to Storage

Keys, SAS tokens, Azure AD data roles, and network rules — the four layers that decide whether a request to a storage account succeeds, and which one to reach for.

Why storage access has its own layer

RBAC decides who can manage the storage *account itself* (create it, resize it, delete it). Getting at the *data inside* — the blobs, files, queues, and tables — runs through a separate set of mechanisms: keys, SAS tokens, Azure AD data roles, and network rules, each answering a different question.



All three have to line up. A valid SAS through a firewall set to **Deny** still fails — and a wide-open network with no valid credential goes nowhere either.

Eight terms you need cold

These cover keys, SAS, and network controls — the trio that almost every storage-access exam question is built around.

Access Key

Two 512-bit keys generated automatically per storage account. Whoever holds one gets full read/write/delete access to every service in the account.

Tip: Treat it like a root password — prefer SAS or Azure AD instead of handing out the key itself.

Shared Access Signature

A signed URL granting time-limited, scoped access to storage resources without revealing the account key itself.

Tip: Always set the shortest expiration the scenario actually needs.

Account SAS

A SAS covering one or more services in the account (blob, file, queue, table), with broad permissions like creating containers.

Tip: Signed with the account key — regenerating that key invalidates every Account SAS tied to it.

Service SAS

A SAS scoped to one specific resource: a single blob, container, queue, table, or file share.

Tip: The most common SAS type — exactly the access needed, nothing more.

User Delegation SAS

A SAS signed with Azure AD credentials instead of the account key, valid for Blob storage only.

Tip: The most secure SAS type — no account key to leak, and the signer's own RBAC role caps what it can do.

Stored Access Policy

A server-side policy on a container, queue, table, or share defining the permissions and expiry that SAS tokens referencing it follow.

Tip: Without one, a SAS can't be revoked early — your only option is rotating the whole account key.

Storage Firewall

Network rules — IP ranges or virtual networks — controlling which networks can even reach the account's public endpoint, separate from authentication entirely.

Tip: With the default action set to Deny, even a valid key or SAS won't get you in from an unlisted network.

Private Endpoint

A network interface with a private IP inside your VNet, connecting directly to the storage account and removing it from the public internet entirely.

Tip: Different from a VNet service endpoint, which keeps traffic on the Azure backbone but leaves the public endpoint reachable unless the firewall is also locked down.

Storage data roles cheat sheet

ROLE	GRANTS	NOTES
Storage Blob Data Owner	Full access to blob data, including setting POSIX ACLs	Highest privilege
Storage Blob Data Contributor	Read, write, and delete blob data	No ACL management
Storage Blob Data Reader	Read blob data only	—
Storage Blob Data Delegator	Get a user delegation key to issue User Delegation SAS tokens	No direct data access

Three flavors of SAS, side by side

TYPE	SCOPE	SIGNED BY	BEST FOR
Account SAS	One or more services account-wide	Account key	Broad, scripted access across resources
Service SAS	A single resource	Account key	Sharing one specific object with exact permissions
User Delegation SAS	Blob storage only	Azure AD identity	The most secure option — tied to an identity, not a shared secret

EXAM GOTCHA

Regenerating an account key **immediately invalidates** every Account SAS and Service SAS signed with it. A **User Delegation SAS** is unaffected, since it was never signed with the account key in the first place.

WHEN IN DOUBT

Skip SAS and keys altogether when you can — grant access with **Azure AD + an RBAC data role** instead. Every request ties back to a real identity and shows up in the audit log.

Granting access to data in a storage account

Walk through this whenever something new needs to read or write blobs, files, queues, or tables.



YES

Attach a Stored Access Policy so it can be revoked server-side

NO

A standalone SAS with a short expiry is fine

DECIDE

Should this storage account be reachable from the public internet at all?

NEVER

Private Endpoint + default action Deny

ONLY KNOWN NETWORKS

Firewall rules (IP / VNet) + default action Deny

BROADLY (RARE)

Leave default Allow — not recommended for production

VERIFY

Confirm the network path is allowed, then confirm the SAS / key / Azure AD role grants the right permission

DONE

Access configured: right identity, right scope, right network path

Five ways to configure storage access

Keys, SAS tokens, network rules, and role assignments can all be set through any of these — pick based on whether it's exploratory, scripted, or part of a repeatable deployment.

Azure Portal

BEST FOR: one-off SAS generation, visual firewall setup

The "Shared access signature" blade builds a SAS URL with a form — useful for understanding exactly which fields make up the signature before scripting it.

```
Storage Account → Access keys  
→ Shared access signature →  
Networking
```

Azure CLI

BEST FOR: scripting, CI/CD pipelines

Generates SAS tokens, lists keys, and updates firewall rules in commands that drop straight into a deployment script.

```
az storage account keys list \  
  --account-name "mystorageacct" --  
  resource-group "rg"  
  
az storage container generate-sas \  
  --account-name "mystorageacct" --  
  name "container" \  
  --permissions r --expiry 2026-12-  
  31  
  
az storage account update \  
  --name "mystorageacct" --default-  
  action Deny
```

> Azure PowerShell

BEST FOR: Windows-centric / enterprise scripting

Fits naturally next to the rest of the Az PowerShell module for teams already standardized on it.

```
Get-AzStorageAccountKey -
ResourceGroupName "rg" `
-Name "mystorageacct"

New-AzStorageContainerSASToken -
Name "container" `
-Permission r -ExpiryTime (Get-
Date).AddDays(7) `
-Context $ctx

Update-
AzStorageAccountNetworkRuleSet `
-ResourceGroupName "rg" -Name
"mystorageacct" `
-DefaultAction Deny
```

<> Bicep / ARM

BEST FOR: infrastructure as code, GitOps

Network rules and role assignments deploy as part of the same template as the storage account itself — nothing left to configure manually after the fact.

```
resource sa
'Microsoft.Storage/storageAccounts@
2023-01-01' = {
  name: 'mystorageacct'
  properties: {
    networkAcls: {
      defaultAction: 'Deny'
      ipRules: [{ value:
'203.0.113.0/24' }]
    }
  }
}
```

🌐 REST API

BEST FOR: custom tooling

Every tool above calls this underneath — useful when building your own integrations or SDK wrappers.

```
GET
https://management.azure.com/{scope
}/
providers/Microsoft.Storage/storage
Accounts/
{name}/listKeys
?api-version=2023-01-01
```


TOOL	BEST FOR	REPEATABLE	LEARNING CURVE
Portal	One-off SAS generation, firewall setup	No	Low
CLI	Scripting / CI-CD	Yes	Medium
PowerShell	Windows / enterprise scripting	Yes	Medium
Bicep / ARM	IaC, GitOps	Yes	Medium–High
REST API	Custom integrations	Yes	High

Principles worth internalizing

Prefer identity over secrets, keep SAS tokens short-lived, and don't rely on authentication alone — lock down the network too.

MECHANISM	TRAFFIC PATH	STILL PUBLICLY ADDRESSABLE?
VNet Service Endpoint	Stays on the Azure backbone	Yes — firewall just restricts the source
Private Endpoint	Private IP directly inside your VNet	No — can be fully unreachable from the internet

EXAM GOTCHA

A VNet **service endpoint** does not make a storage account private — the public endpoint still resolves and is still reachable in principle; the firewall rule is what restricts the source. Only a **private endpoint** can make the account fully unreachable from the public internet.

Six mistakes that show up on the exam

Embedding account keys in client code

A leaked key hands over full read/write/delete access to the entire account — never ship one inside an app.

Issuing a SAS with no real expiration

A far-future or missing expiry turns a "temporary" token into a permanent credential nobody is tracking.

Forgetting key rotation breaks key-signed SAS

Regenerating an account key silently invalidates every Account SAS and Service SAS tied to it.

Assuming a service endpoint means private

It keeps traffic on the Azure backbone, but the account is still publicly addressable unless the firewall — or a private endpoint — says otherwise.

Granting Owner-level data roles by default

Storage Blob Data Contributor or Reader usually covers the need — Owner also grants ACL management most apps never use.

Leaving the default network action as Allow

In production, default to Deny and explicitly allow the networks that actually need access.